SolarDB Pascual: gobernanza de datos, automatización ETL e IoT sobre un repositorio relacional

Juan José Mazo Rodríguez
Institución Universitaria Pascual Bravo · Bases de Datos I (SD1006), Grupo 811
Docente: Ramiro Grisales Montoya
9 de octubre de 2026

Parte A. Consulta

Bloque 1. Gobernanza de datos

G1. Data governance, data management y data quality

La gobernanza de datos define quién decide sobre los datos, con qué reglas y con qué responsabilidades, y orienta a las demás áreas de la gestión de datos (DAMA International, 2017). La gestión de datos ejecuta esas reglas (modelar, almacenar, integrar y proteger) y la calidad de datos mide si los datos sirven para su propósito. En SolarDB, gobernar es acordar que toda lectura tenga dispositivo, hora y unidades documentadas; gestionar es crear las tablas, la carga y los permisos; y la calidad es verificar que la potencia no sea negativa ni falten lecturas.

G2. Data owner, data steward y data custodian

El data owner responde por un conjunto de datos y autoriza su uso; el data steward mantiene los significados, las reglas de calidad y el diccionario; y el data custodian protege y opera técnicamente los datos sin decidir sobre su contenido (DAMA International, 2017). En SolarDB, el owner sería el coordinador del proyecto, el steward el equipo que mantiene el diccionario y el custodian quien administra PostgreSQL con el rol solar_admin. Así se separan quien decide, quien cuida la calidad y quien opera.

G3. Metadatos y catálogo de datos

Los metadatos son datos que describen otros datos, y el catálogo de datos es su inventario consultable (DAMA International, 2017). Al diccionario del Taller 1 le faltarían cuatro columnas para funcionar como catálogo gobernado: responsable, origen, regla de calidad y fecha de actualización. Sin ellas se conoce la estructura de la base, pero no la procedencia del dato, quién responde por él ni cuándo se revisó.

G4. Dimensiones de calidad de datos

La completitud verifica que estén los datos esperados; la validez, que cumplan formato y rango; la unicidad, que no se repitan; la consistencia, que no se contradigan entre tablas; la oportunidad, que lleguen a tiempo; y la exactitud, que reflejen la realidad (DAMA International, 2017). Cuatro se traducen en restricciones de PostgreSQL (PostgreSQL Global Development Group, s. f.):

Completitud: dispositivo_id INT NOT NULL, ts TIMESTAMPTZ NOT NULL
Validez: CHECK (irradiancia BETWEEN 0 AND 1500)
Unicidad: PRIMARY KEY (dispositivo_id, ts)
Consistencia: FOREIGN KEY (dispositivo_id) REFERENCES dispositivo (id)

G5. Principio de mínimo privilegio

El mínimo privilegio consiste en dar a cada usuario o proceso solo los permisos indispensables, para limitar el daño de un error o de una credencial filtrada. En PostgreSQL se aplica con CREATE ROLE, GRANT y REVOKE (PostgreSQL Global Development Group, s. f.). Para SolarDB se diseñaron tres roles, cuyos permisos se resumen en la Tabla 1; el script completo está en sql/03_roles.sql.

Tabla 1. Permisos de cada rol sobre las tablas de SolarDB

Rol	stg_lectura_raw	lectura_demo	etl_log
solar_ingesta	SELECT, INSERT, DELETE, TRUNCATE	SELECT, INSERT	SELECT, INSERT, UPDATE
solar_lector	Ninguno	SELECT	Ninguno
solar_admin	ALL	ALL	ALL

G6. Ley 1581 de 2012 y postura ética

La Ley 1581 de 2012 define dato personal como la información asociada a una persona natural determinada o determinable (Congreso de la República de Colombia, 2012). La telemetría de la planta describe equipos, no personas, por lo que no es dato personal. Registrar a los operadores que atienden las alarmas sí lo sería, y exigiría autorización del titular, finalidad informada y medidas de seguridad. El equipo se compromete a recoger solo lo necesario, usar identificadores seudonimizados, restringir el acceso por rol y no publicar datos personales en el repositorio.

Bloque 2. Automatización de procesos ETL

E1. ETL frente a ELT

En ETL los datos se transforman antes de cargarse (Kimball y Caserta, 2004); en ELT se cargan primero y se transforman dentro del destino. SolarDB se parece más a ELT: el mensaje crudo se guarda en la columna payload (JSONB) de staging y luego una sentencia SQL extrae sus atributos. Conservar el original permite repetir la transformación sin pedir de nuevo el dato al dispositivo.

E2. Carga completa, carga incremental e idempotencia

La carga completa vuelve a cargar todo el conjunto; la incremental procesa solo lo nuevo o modificado. Una carga es idempotente si ejecutarla varias veces con la misma entrada deja el mismo resultado. INSERT ... ON CONFLICT lo logra con la clave primaria de la tabla lectura: si ya existe la pareja (dispositivo_id, ts), DO NOTHING omite la fila sin error (PostgreSQL Global Development Group, s. f.).

E3. Formas de programar una carga

Tabla 2. Comparación de cron, pg_cron y Apache Airflow

Herramienta	Qué es	Cuándo conviene	Complejidad
cron / Programador de tareas	Planificador del sistema operativo que lanza un comando en horarios fijos	Un script independiente con periodicidad simple	Baja
pg_cron	Extensión de PostgreSQL que programa sentencias SQL (Citus Data, s. f.)	Tareas que son solo SQL	Baja a media
Apache Airflow	Plataforma de orquestación de flujos con dependencias y monitoreo (Apache Software Foundation, s. f.)	Muchas tareas encadenadas y varias fuentes	Alta

En este proyecto se usaría cron o el Programador de tareas, porque el flujo es un único comando y no requiere instalar nada en el servidor.

E4. Bitácora del proceso ETL

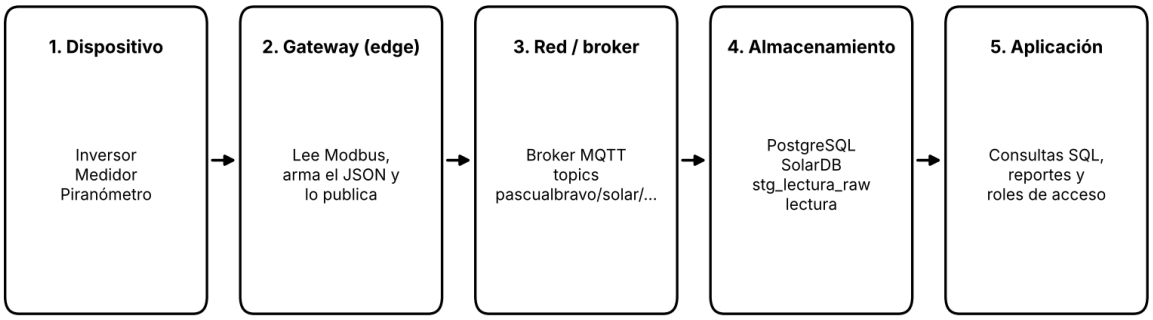
La bitácora registra el identificador de la ejecución, el proceso, el origen, la hora de inicio y fin, las filas leídas, cargadas, duplicadas y rechazadas, el estado y el error. Es una herramienta de gobernanza porque da trazabilidad (data lineage): permite saber de dónde vino cada dato, cuándo entró y con qué resultado (DAMA International, 2017). En SolarDB se guarda en la tabla etl_log.

Bloque 3. Internet de las Cosas (IoT)

I1. Arquitectura IoT por capas

La arquitectura tiene cinco capas (Figura 1): el dispositivo (inversor, medidor y piranómetro) mide; el gateway lee los equipos y publica los datos como JSON; el broker MQTT los distribuye (OASIS, 2019); el almacenamiento, donde está SolarDB, los guarda; y la aplicación los consulta. En este trabajo el simulador reemplaza a los dispositivos y al gateway.

Figura 1. Arquitectura IoT por capas de SolarDB



I2. MQTT

MQTT es un protocolo publish/subscribe: los publicadores envían mensajes a un topic y los suscriptores reciben los de los topics a los que se suscribieron, siempre a través de un broker (OASIS, 2019). QoS 0 entrega como máximo una vez, QoS 1 al menos una vez y QoS 2 exactamente una vez. Para SolarDB conviene QoS 1, porque los duplicados los absorbe la carga idempotente. Jerarquía propuesta:

pascualbravo/solar/<sitio>/<dispositivo>/<variable>
pascualbravo/solar/sede-principal/inversor-01/p_ac
pascualbravo/solar/sede-principal/piranometro-01/irradiancia

I3. Modbus, MQTT y OPC UA

Tabla 3. Comparación de Modbus, MQTT y OPC UA

	Modbus (RTU/TCP)	MQTT	OPC UA
Comunicación	Maestro/esclavo, petición y respuesta (Modbus Organization, s. f.)	Publish/subscribe con broker (OASIS, 2019)	Cliente/servidor y publish/subscribe (OPC Foundation, s. f.)
Uso típico	Leer registros de inversores y medidores	Enviar telemetría del gateway al servidor	Interoperar equipos y sistemas industriales
Datos	Registros binarios de 16 bits	Carga libre (JSON, texto, binario)	Modelo de información tipado y autodescriptivo
Seguridad	Casi ninguna de forma nativa	TLS, usuario y contraseña, permisos en el broker	Integrada: certificados, firma y cifrado

I4. Payload JSON del inversor y consultas JSONB

Mensaje que un inversor publicaría cada 5 minutos, con ocho campos y las unidades en el nombre:

```
{
"device_id": 1, "ts": "2026-10-05T06:00:00-05:00",
"p_ac_kw": 3.197, "p_dc_kw": 3.350, "v_dc_v": 412.5,
"i_dc_a": 8.1, "temp_inversor_c": 41.2,
"frecuencia_hz": 60.0, "estado": "RUN"
}
```

Consulta 1, con -> y ->>, extrae atributos y filtra por potencia:

```
SELECT payload->>'device_id' AS dispositivo, (payload->>'p_ac_kw')::numeric AS p_ac_kw,
    payload->'estado' AS estado
FROM stg_lectura_raw WHERE (payload->>'p_ac_kw')::numeric > 3;
```

Consulta 2, con ? y @>, cuenta las alarmas y los equipos en marcha:

```
SELECT count(*) FILTER (WHERE payload ? 'alarma') AS con_alarma,
    count(*) FILTER (WHERE payload @> '{"estado":"RUN"}') AS en_marcha
```

FROM stg_lectura_raw;

I5. Volumen de datos, particionamiento y BRIN

Con 10 dispositivos, 5 variables y una lectura por minuto se generan 10 × 5 × 60 × 24 × 30 = 2 160 000 filas al mes. El particionamiento por rango divide la tabla lectura por intervalos de ts, de modo que las consultas por fecha leen pocas particiones y los datos antiguos se archivan descartando una partición. El índice BRIN guarda el mínimo y el máximo de cada grupo de bloques; como las lecturas llegan ordenadas en el tiempo, ocupa poco espacio y acelera los rangos de fechas (PostgreSQL Global Development Group, s. f.). TimescaleDB, una extensión de PostgreSQL, automatiza el particionamiento y la compresión (Timescale, s. f.).

Bloque 4. Control de versiones

V1. Git, GitHub y credenciales

Git es el sistema de control de versiones local; GitHub es una plataforma que aloja repositorios Git (Chacon y Straub, 2014). Un repository es el proyecto con su historial; un commit, una instantánea de cambios; una branch, una línea de desarrollo paralela; un pull request, una propuesta de fusionar una rama tras revisarla; y .gitignore, la lista de archivos que no se versionan. Las credenciales no se suben porque el historial es permanente y los repositorios públicos son rastreados; se evita usando variables de entorno o un archivo .env excluido en .gitignore.

Parte B. Práctica guiada: primer pipeline de ingesta

Se trabajó con PostgreSQL 16 y Python 3. El código completo está en el repositorio (carpetas etl/ y sql/).

Paso 1. Simulador IoT

Se modificó etl/simulador.py para generar lecturas de dos dispositivos (144 lecturas cada uno, una cada 5 minutos) y se agregó el campo voltaje_dc (V). El resultado es data/lecturas.jsonl, con 288 líneas.

Paso 2. Staging y tabla destino

El script sql/01_tablas.sql crea stg_lectura_raw, lectura_demo y la bitácora etl_log. Para cargar el archivo se eligió \copy de psql porque carga todo de una vez, es más rápido que insertar fila por fila y no exige instalar paquetes adicionales. Se usa el formato CSV con delimitador y comilla poco comunes para que el JSON entre intacto.

Paso 3. Transformación y carga

La sentencia de sql/02_carga.sql extrae los atributos del payload con ->>, los convierte al tipo correcto y omite las lecturas ya existentes:

```
INSERT INTO lectura_demo (dispositivo_id, ts, p_ac, irradiancia, temp_modulo, payload)
SELECT (payload->>'device_id')::int, (payload->>'ts')::timestamptz,
    (payload->>'p_ac')::numeric(10,3), (payload->>'irradiancia')::numeric(8,1),
```

```
    (payload->>'temp_modulo')::numeric(5,1), payload
FROM stg_lectura_raw ORDER BY id
ON CONFLICT (dispositivo_id, ts) DO NOTHING;
```

Paso 4. Prueba de idempotencia

El flujo se ejecutó dos veces con el mismo archivo. Después de la primera ejecución, SELECT COUNT(*) FROM lectura_demo devolvió 288, y después de la segunda también 288. En la segunda corrida las 288 lecturas ya existían, así que ON CONFLICT DO NOTHING las omitió; por eso la carga es idempotente.

Paso 5. Bitácora y automatización

La tabla etl_log recibe una fila por ejecución con las filas leídas, cargadas, duplicadas y rechazadas, el estado y el error. Todo el flujo se lanza con un solo comando: python etl/run_etl.py. La programación cada hora quedaría así:

cron (Linux): 0 * * * * cd /ruta/solardb && python etl/run_etl.py >> etl.log 2>&1
Windows: schtasks /create /tn "SolarDB ETL" /sc hourly /tr "python C:\ruta\solardb\etl\run_etl.py"

Paso 6. Gobernanza aplicada

El script sql/03_roles.sql crea el rol solar_lector con permiso de solo lectura sobre lectura_demo. Con SET ROLE solar_lector, el SELECT funciona y un INSERT es rechazado con el error permission denied for table lectura_demo.

Parte C. Repositorio en GitHub

El repositorio está disponible en https://github.com/JUANMAZO398/solardb-Mazo-Rodriguez Contiene README.md, .gitignore, .env.example y las carpetas docs/, data/, etl/ y sql/. Las credenciales no se suben: la conexión usa variables de entorno o un archivo .env excluido en .gitignore.

Referencias

Apache Software Foundation. (s. f.). Apache Airflow documentation. https://airflow.apache.org/docs/

Chacon, S., y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book

Citus Data. (s. f.). pg_cron [Software]. GitHub. https://github.com/citusdata/pg_cron

Congreso de la República de Colombia. (2012). Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales. http://www.secretariasenado.gov.co/senado/basedoc/ley_1581_2012.html

DAMA International. (2017). DAMA-DMBOK: Data management body of knowledge (2.ª ed.). Technics Publications.

Kimball, R., y Caserta, J. (2004). The data warehouse ETL toolkit. Wiley.

Modbus Organization. (s. f.). Modbus specifications. https://modbus.org/specs.php

OASIS. (2019). MQTT version 5.0. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

OPC Foundation. (s. f.). OPC unified architecture. https://opcfoundation.org/about/opc-technologies/opc-ua/

PostgreSQL Global Development Group. (s. f.). PostgreSQL documentation. https://www.postgresql.org/docs/current/

Timescale. (s. f.). TimescaleDB documentation. https://docs.timescale.com/